

COMP0227 Lab 01: Features in Images and in SLAM

5th October, 2026

1 Scope and Purpose

Like ORB-SLAM, pySLAM implements a sparse SLAM system which consists of individual map points with feature descriptors. By matching descriptors over multiple frames, points can be tracked over time and the measurements can then be triangulated to work out 3D position. As briefly discussed in COMP0249, there are many types of feature descriptors. Some were designed by hand, such as ORB and SIFT. Others were learned from data, such as SuperPoint and LightGlue.

This lab has two main purposes. The first is to extend the operation of pySLAM to your machines by introducing some features implemented in PyTorch. This gives us the further opportunity to test implementation issues. The second is to carry out a small investigation into the function and role of feature points. Specifically:

1. How much do the different features differ when you look at them one pair of images at a time?
2. Do these differences have any practical effect on SLAM?

You will first match features between pairs of images. You will then watch SLAM run with different features. Finally you will run SLAM several times with each and compare the results.

This lab assumes that you have completed Lab 00, and that pySLAM is installed and working. As before, run all the commands from the `pyslam` folder.

2 The Features

We will use four sets of features listed in Table 1. The names are the ones pySLAM uses.

Note that `SUPERPOINT` and `LIGHTGLUE` use the same feature detector / extractor code. They differ only in how the features between pairs of images are matched.

pySLAM supports many more features. The full list can be found by running the command:

```
pixi run feature-matching --list
```

Name	Detector and descriptor	Matcher
ORB2	ORB, as used in ORB-SLAM2: hand-designed, binary	nearest neighbour
ROOT_SIFT	SIFT with RootSIFT descriptors: hand-designed	nearest neighbour
SUPERPOINT	SuperPoint: learned	nearest neighbour
LIGHTGLUE	SuperPoint: learned	LightGlue: learned

Table 1: The minimal set of feature points used in this lab. pySLAM supports many more, which you are welcome to investigate with.

This contains the full list. This includes most of the hand-designed ones which appear in OpenCV¹, together with a set of machine learned ones. The weights for the machine learned features have to be downloaded, and the list will include the command which installs the models of the learned ones. Note that some of the models are very large, or come from sites which can be very flaky. Feel free to investigate the other feature sets, but we would recommend doing this after you've completed the main activities in this module.

3 Activities

Activity 1: Match One Pair of Images

Run:

```
pixi run feature-matching --features ORB2 --test graf
```

This finds up to 2000 features in each of two photographs of a wall which were taken from different directions, and matches them. Because the wall is flat, the two images are related by a homography. The software fits one to the matches. The matches which agree with it are the *inliers*; the others are wrong.

Four windows open. Two show the matches: all of them, and the inliers only (Figure 1). The blue outline is where the first image lies in the second, according to the homography. The other two windows are histograms of the distances between the descriptors of the matched features. Press Enter in the terminal to close the windows.

The last line printed in the terminal before you press Enter is a summary:

```
Summary: ORB2 on graf: 2000 and 2000 keypoints, 81 matches,
56 inliers (69% of the matches, homography), 65 ms (detection,
description and matching)
```

Your numbers, especially the time, will be different.

- Where in the images are the matches? Where are there none?
- Look at the matches which are not inliers. Can you see why the matcher was fooled?
- Does the blue outline look right?

¹SURF is not included due to licensing issues.

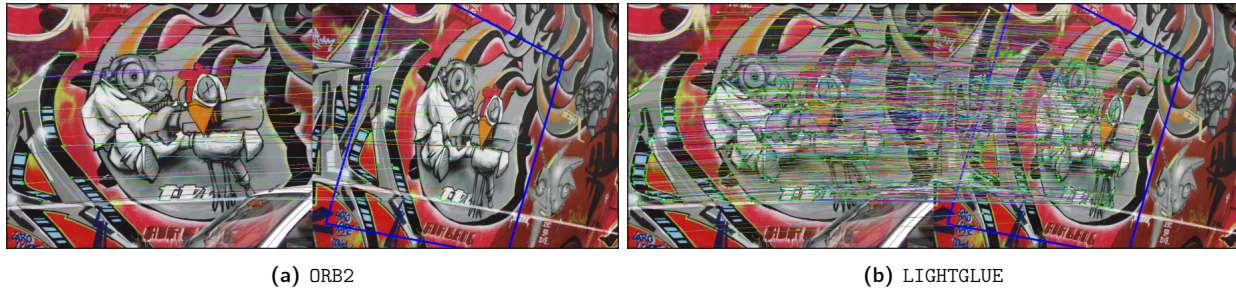


Figure 1: The inlier matches on the `graf` pair with two of the sets of features.

Activity 2: Compare the Features on the Same Pair

Run the same command with each of the four sets of features, and fill in the table from the summary lines.

	Keypoints	Matches	Inliers	Inliers (%)	Time (ms)
ORB2					
ROOT_SIFT					
SUPERPOINT					
LIGHTGLUE					

- Which features give the most inliers? Which give the highest percentage of inliers? Are they the same?
- The software asked for 2000 features in each image. Did it get them from every detector? Why might a detector return fewer?
- `SUPERPOINT` and `LIGHTGLUE` start from the same keypoints and descriptors. What does the learned matcher change, and what does it cost?
- Which is the fastest? The video in Lab 00 has 10 frames per second. Are any of them too slow for that on your computer? (The time is for two images.)
- If you could only keep one number from this table to choose features for SLAM, which would it be? You can test your guess in Activity 5.

Activity 3: Harder and Easier Pairs

The `graf` pair has a large change of viewpoint but the images have very large, clear and visually distinct features. This is not representative of most cases. First, in SLAM the two frames are often taken a fraction of a second apart, so the change in viewpoint is much harder. Second, scenes often much more crowded, or contain far fewer clear and distinct features. Table 2 lists some other images pairs which are provided. Use `--test` to choose different pairs.

For example:

```
pixi run feature-matching --features SUPERPOINT --test kitti_step
```

Name	The two images
kitti_step	two frames of the KITTI video, half a second apart
kitti_LR	the left and right images of the KITTI stereo camera
graf	a wall, from two directions
churchill	a building, from two viewpoints
box	a box, and the same box among other objects
mars	two images of the surface of Mars (very hard)

Table 2: Other image pairs to test with

For the two KITTI pairs the scene is not flat, so the software fits a fundamental matrix instead of a homography.

- Repeat the comparison of Activity 2 on `kitti_step`. Is the difference between the features as large as it was on `graf`? What does this suggest about SLAM on a video?
- Try `mars` and `churchill`. Which features still work, and which fail? When one fails, what does the summary say?
- Add `--num-features 500`. What changes? What would you expect it to change in SLAM?

You can also match two images of your own:

```
pixi run feature-matching --features ORB2 --images first.jpg second.jpg
```

Take two photographs of the same thing with your phone and try it. Then make it harder: move further round, move closer, rotate the phone, or change the lighting. With your own images the software fits a fundamental matrix; add `--model homography` if the scene is flat.

- Can you find a pair on which the hand-designed features fail and the learned ones work? And the other way round?

Activity 4: Watch SLAM with Different Features

In Lab 00 SLAM used ORB2. The option `--features` changes this:

```
pixi run slam --features SUPERPOINT
```

Run it with `SUPERPOINT`, and then with one of the other sets. Compare what you see with what you saw in Lab 00. Remember that `q` or `Esc` in the Camera window quits. Furthermore the throttling mechanism mentioned in Lab 00 might reduce the speed with which frames are delivered to ensure that good tracking results are obtained.

- Camera: are the features in the same places as the ORB features were? Are more or fewer of them matched to the map? Are the trails longer or shorter?
- # matches: how does the number of matches compare with ORB?
- timing: how long does tracking take for each frame? The camera produces a frame every 0.1 seconds.
- Map Viewer: does the map have more points or fewer? How many keyframes are there?

- Is the loop still closed at the end of the video?

Now run it with `LIGHTGLUE`. On some computers this is too slow for the frame rate of the camera, and tracking is lost: the terminal says so, and suggests `--throttle`, which slows the video down when tracking gets weak. If this happens to you, try it:

```
pixi run slam --features LIGHTGLUE --throttle
```

- In Activity 2, `LIGHTGLUE` probably gave you the most matches. Did that make it the best in SLAM?

Activity 5: Compare the Features in SLAM

Watching one run is not a fair comparison. You saw in Lab 00 that two runs with the same features give different errors. To compare features we must run each several times.

The following command runs SLAM, without windows, three times with each of three sets of features: nine runs of about two minutes, so about twenty minutes in all. Close other programs first, and do not run anything else while it is working.

```
pixi run slam-evaluation --features ORB2 ROOT_SIFT SUPERPOINT --runs 3
```

While it runs, carry on thinking about the questions of the earlier activities.

At the end it prints a set of tables, and writes them with a report to a folder `results/eval_<date>`. Open the file `report.html` in that folder in a web browser. There is one column for each set of features. The tables are:

- `rmse` and `max`: the trajectory error in metres, as in Lab 00, over the frames which were tracked.
- `percent_tracked`: the percentage of the frames which have a position in the final trajectory.
- `percent_lost`: the percentage of the frames in which tracking was lost.

Each table gives the average over the runs and the standard deviation.

- Which features have the smallest average error? Is the difference between the features larger or smaller than the difference between runs with the same features?
- From these runs, can you say that one set of features is better than another? How many runs would you need?
- Compare the order of the features here with the one you found in Activity 2. Was your guess right?
- On `graf` the features differed by a large factor in the number of inliers. Why might such a large difference in a pair of images make so little difference in SLAM? Think about what you found on `kitti_step`, and about what else SLAM does with the features.

If you have time, add `LIGHTGLUE`:

```
pixi run slam-evaluation --features LIGHTGLUE --runs 2
```

- Look at `percent_tracked` before you look at `rmse`. If tracking was lost, what does the error mean? Can a run which lost track have a smaller error than one which did not?
- The tables name the “best” features for each measure. Do you agree with them?
- Run it again with `--throttle`. What is this comparison now measuring, and what has it stopped measuring?

4 Extra Activities

These are optional.

Activity 6: A Standard Dataset

KITTI is a car on a road. The TUM RGB-D sequences are a hand-held camera indoors, which is a very different problem: the motion is less regular, and there is less texture. If you have downloaded a TUM sequence, for example `rgbd_dataset_freiburg1_xyz`, you can watch SLAM run on it with:

```
pixi run slam --tum path/to/rgbd_dataset_freiburg1_xyz \
  --settings settings/TUM1.yaml
```

and evaluate on it with:

```
pixi run slam-evaluation --tum path/to/rgbd_dataset_freiburg1_xyz \
  --settings settings/TUM1.yaml --features ORB2 SUPERPOINT
```

The folder must contain the ground truth (`groundtruth.txt`). The frames are taken from the file `associations.txt` if there is one, and from `rgb.txt` if not. A sequence as it is downloaded has no `associations.txt`, so it runs as monocular SLAM: the depth images are not used.

- Do the features come out in the same order as on KITTI?
- The errors are in metres. How do they compare with the ones on KITTI? Why are they so different?

Activity 7: Your Own Video

You can run SLAM on a video of your own, or on a folder of images. You should be able to directly use your data you collected from COMP0249 if you still have it available.

1. The Settings File

pySLAM needs to know the calibration of your camera. You should be able to use the `yaml` file generated by the scripts from COMP0249. However, if you wish to edit the file manually:

- the focal lengths (`Camera.fx`, `Camera.fy`) and the principal point (`Camera.cx`, `Camera.cy`), in pixels;
- the distortion (`Camera.k1`, `Camera.k2`, `Camera.p1`, `Camera.p2`, `Camera.k3`);
- the size of the image (`Camera.width`, `Camera.height`) and the frame rate (`Camera.fps`).

The settings must match the video. If you change the resolution of the video, the calibration changes with it.

If you already have a settings file for ORB-SLAM2 for your camera, you can use it as it is: pySLAM reads the same camera parameters. An ORB-SLAM2 file does not give the size of the image; pySLAM then takes it from the video or the images, and says so in the terminal. The `ORBextractor` and `Viewer` parameters in the file are ignored.

2. Watch SLAM Run

For a video:

```
pixi run slam --video my/video.mp4 --settings settings/MY_CAMERA.yaml
```

For a folder of images, which are read in the order of their names:

```
pixi run slam --images my/frames --pattern "*.jpg" --fps 30 \
  --settings settings/MY_CAMERA.yaml
```

Here `--pattern` says which files to read (without it, `*.png`) and `--fps` is the frame rate (without it, 10 frames per second). You can add `--features` to both, as in Activity 4.

Some things to know:

- The video does not have to be an `mp4` file. Videos are read with FFmpeg, and `mov` (including the HEVC videos of an iPhone), `mkv`, `m4v`, `avi` and `webm` files have all been tested.
- A phone stores every video the same way round, with a note saying how the phone was held. pySLAM applies this note, as a video player does: the frames are the right way up. A video filmed with the phone upright therefore has upright frames, for example 1080 pixels wide and 1920 high. The settings file must describe the frames as they are shown: `Camera.width` and `Camera.height` are those of the upright frame, and so are the focal lengths and the principal point. A calibration made with the phone held the other way does not apply as it is.
- Phones often record at a frame rate which varies. pySLAM uses the single frame rate which the file reports.
- The images can be PNG, JPEG, BMP, TIFF or WebP files. The HEIC photographs of an iPhone cannot be read: export them as JPEG first.

3. Compare the Features

The evaluation takes the same options:

```
pixi run slam-evaluation --video my/video.mp4 \
  --settings settings/MY_CAMERA.yaml --features ORB2 SUPERPOINT
```

Without ground truth there is no trajectory error, only `percent_tracked` and `percent_lost`.

- Without a trajectory error, how could you decide which features did better on your video?
- What in your video made tracking difficult? Think about how fast the camera moved, what it was looking at, and the lighting.